Documento de análisis — Club Deportivo Séneca (Entrega 2)

Samuel Caro, David Arismendy, Jose Luis

1. Introducción

Este documento explica cómo diseñamos el Club Deportivo Séneca para la Entrega 2. Partimos del análisis de la Entrega 1 (guiandonos con las reglas del negocio y el primer UML) y de los comentarios que nos hicieron en la primera sustentacion. Con eso ajustamos el UML y después dejamos el código igual al diagrama.

La aplicación es un programa de consola en Java 8 que se abre en Eclipse y se ejecuta desde la clase `Main`. Los datos se guardan en archivos de texto dentro de la carpeta `datos`, que está fuera de la carpeta `src`.

Cómo está organizado este documento:

- Sección 2: qué cambió frente a la Entrega 1.
- Sección 3: estado de la aplicación y funcionalidades que soporta.
- Secciones 4 y 5: UML de alto nivel y UML detallado.
- Sección 6: responsabilidad de cada clase.
- Sección 7: decisiones de diseño y por qué las tomamos (principios SOLID).
- Sección 8: tres diagramas de secuencia de los requerimientos críticos.
- Sección 9: persistencia.
- Sección 10: anexo del uso de inteligencia artificial.

Regla que seguimos al escribir: todo lo que aparece aquí existe en el código. Lo que no quedó implementado lo decimos de frente en la sección 3.

2. Cambios frente a la Entrega 1

Comparamos el UML de la Entrega 1 (versión 1) con el UML final de esta entrega. Los cambios salieron de la retroalimentación que recibimos y de lo que fuimos encontrando al programar. Los 18 requisitos y las reglas del negocio siguen siendo los mismos; lo que cambió es cómo los representamos.

Tema	Entrega 1 (UML v1)	Entrega 2 (UML final)	Por qué lo cambiamos
Clase que coordina	No había. ClubDeportivo solo se nombraba en la matriz de trazabilidad, pero no estaba dibujada.	Existe ClubDeportivo con las listas del club (usuarios, instalaciones, reservas, ventas, etc.) y las operaciones que juntan varios objetos.	Las reglas que involucran varios objetos necesitaban un solo lugar que las coordinara.

Tema	Entrega 1 (UML v1)	Entrega 2 (UML final)	Por qué lo cambiamos
Líneas del UML	Las líneas tenían verbos (“tiene”, “realiza”, “registra”).	Las líneas solo llevan cardinalidad. Cada relación se ve como un atributo dentro de la clase.	Nos lo pidieron en la retroalimentación y así el UML se pasa directo a Java.
Instalaciones	Instalacion era abstracta y tenía cuatro subclases: CanchaConjunto, CanchaPadel, CanchaTenis y MesaTenisMesa.	Una sola clase Instalacion con el atributo tipo (enum TipoInstalacion).	Las subclases solo cambiaban de nombre, no tenían comportamiento propio.
Habilitación de jugadores	Atributo booleano habilitado (InscripcionEquipo) y habilitada (InscripcionCompetencia).	Atributo estado de tipo EstadoInscripcion (ACTIVA, SUSPENDIDA, RETIRADA). También lo tiene InscripcionAcademia.	Un booleano no distingue suspendido de retirado, y así un socio puede estar suspendido en un deporte y activo en otro.
Competencias	tipo (ASCUN o TORNEO_CERROS) y formato (LIGA o GRUPOS_ELIMINACION) eran atributos independientes.	En el UML el formato queda como atributo derivado del tipo (formato) y se agrega la tabla de posiciones (clase PosicionTabla).	La retroalimentación pidió aclarar la diferencia entre ASCUN y Torneo Cerros. Ver sección 7 y la limitación que aclaramos en la sección 3.
Nómina de la competencia	Solo una nota decía que se inscriben jugadores habilitados.	InscripcionCompetencia tiene una nómina (lista de socios) y PartidoOficial la consulta.	Para saber qué jugadores pueden jugar los partidos oficiales del equipo.
Puntos por deporte	No existían.	DeporteConjunto guarda puntosVictoria, puntosEmpate y permiteEmpate, y calcula los puntos de un resultado. PosicionTabla acumula victorias, empates, derrotas y puntos.	Cada deporte da puntos distintos y algunos no permiten empates.

Tema	Entrega 1 (UML v1)	Entrega 2 (UML final)	Por qué lo cambiamos
Posiciones	InscripcionEquipo tenía posicion como texto (String).	Posicion es una clase que DeporteConjunto tiene (composición) e InscripcionEquipo referencia.	Cada deporte tiene sus propias posiciones y así no se escribe cualquier texto.
Descuentos	Venta tenía el atributo tipoDescuento (enum TipoDescuento con NINGUNO, EMPLEADO y CODIGO_COMPARTIDO) y el atributo puntosRedimidos.	Interfaz PoliticaDescuento con tres clases (SinDescuento, DescuentoEmpleado y DescuentoCodigoCompartido). Venta guarda una politicaDescuento. Se eliminaron el enum TipoDescuento y puntosRedimidos.	Para no llenar Venta de if/else y poder agregar descuentos nuevos sin tocarla (patrón Strategy).
Compradores	El comprador era un socio o un empleado.	Interfaz Comprador, que implementan Socio y Empleado. Venta guarda un Comprador.	Venta no necesita saber qué tipo de usuario compra.
Partidos de práctica	Participaban socios.	participantes es una List<Usuario> de 2 a 4, así puede jugar un entrenador. El código no deja jugar al administrador.	El enunciado permite que un entrenador participe en la práctica.
Entrenamiento	La sesión solo tenía fecha, hora, duración y tipo. Sus relaciones (entrenador, instalación, equipo) eran líneas con verbos. EstadoAsistencia tenía ASISTIO, AUSENTE y TARDE.	La sesión guarda su entrenador, su instalación y su equipo o deporte individual como atributos. EstadoAsistencia gana el valor PENDIENTE.	Cuando se agrega un socio a la sesión todavía no se sabe si asistió.
Inventario y sugerencias	MovimientoInventario solo tenía fecha, cantidad y tipo. SugerenciaProducto solo tenía nombre, descripción y estado.	MovimientoInventario guarda producto, origen y destino (opcionales). SugerenciaProducto guarda quién propuso, qué administrador decidió y el producto creado.	Para poder rastrear cada cambio de inventario y cada decisión.

Tema	Entrega 1 (UML v1)	Entrega 2 (UML final)	Por qué lo cambiamos
Turnos	SolicitudCambioTurno guardaba el nuevo día y las nuevas horas.	También guarda el turno original y el administrador que decide. ClubDeportivo revisa la cobertura mínima antes de aprobar.	La regla de cobertura necesita contar a todo el personal, algo que un solo empleado no puede hacer.
Impuestos y propina	Estaban escritos en notas del diagrama (IVA 19 %, impuesto al consumo 8 %, propina sugerida 10 %).	Son constantes en el código: IVA en VentaTienda; IMPUESTO_CONSUMO y PROPINA_SUGERIDA en VentaCafeteria.	Para que el valor esté en el código y no solo en un texto.
Colecciones	Se usaba ArrayList (por ejemplo alergenos: ArrayList<String>).	Se usa List.	La guía del curso recomienda programar contra la interfaz.
Presentación del UML	Un diagrama general con atributos y figuras por tema.	UML de alto nivel (solo clases y relaciones) y UML detallado en páginas por paquete.	Para que cada diagrama se lea bien.

Lo que **no** cambió: la jerarquía de usuarios (Usuario, con Socio, Empleado y Administrador, y Empleado con Entrenador y Fisioterapeuta), la jerarquía de productos (ArticuloTienda, Bebida y Snack), los tres tipos de partido, las estadísticas por deporte, VentaTienda y VentaCafeteria con su composición a DetalleVenta, y las enumeraciones de la Entrega 1 (Categoria, Nivel, Modalidad, TipoSesion, EstadoSolicitud, TipoMovimiento, entre otras).

3. Estado de la aplicación y funcionalidades

3.1 Cómo funciona hoy

Al ejecutar `Main` se crea la consola (`ConsolaClubDeportivo`), que carga los archivos de la carpeta `datos`, muestra un menú de 42 opciones y, al salir con la opción 0, guarda todo otra vez. La consola es un borrador: sirve para ejecutar y demostrar cada funcionalidad, pero todavía no tiene inicio de sesión ni menús distintos por tipo de usuario. Los usuarios sí tienen login y contraseña (se piden al registrarlos y se guardan), pero la consola aún no los usa para entrar.

Si no hay archivos de datos, la aplicación arranca vacía y se puede llenar desde el menú: la opción 6 crea la instalación si no existe y la opción 4 crea el producto y la ubicación si no existen.

3.2 Qué soporta cada requisite (referidos mas Adelante como HU)

Esta tabla actualiza la matriz de trazabilidad de la Entrega 1. Dice qué clases y métodos hacen cada historia y en qué opción del menú se prueba.

Requisito	Clases y métodos principales	Menú	Estado
1 Consultar disponibilidad	ClubDeportivo.consultarDisponibilidad, Instalacion.estaDisponible	6	Parcial: se usa al reservar, pero no hay una opción solo para consultar.
2 Reservar una instalación	ClubDeportivo.registrarReserva, Reserva, Instalacion, DeporteIndividual.permiteNuevaReserva	6, 8	Completo
3 Inscribir socio en equipo	ClubDeportivo.inscribirSocioEnEquipo, Equipo, InscripcionEquipo	14, 15, 17, 18	Completo (no valida la edad contra la categoría)
4 Inscribir socio en academia	ClubDeportivo.inscribirSocioEnAcademia, InscripcionAcademia, DeporteIndividual	13, 16, 18	Completo
5 Consultar información deportiva	Equipo, Partido, EstadisticaJugador, InscripcionAcademia	17, 18, 25, 30, 42	Completo
6 Programar sesión de entrenamiento	ClubDeportivo.programarSesion, SesionEntrenamiento	26, 30	Completo (no revisa el turno del entrenador)
7 Asistencia y evaluación	ClubDeportivo.registrarAsistencia, RegistroAsistencia	27, 28, 29, 30	Completo
8 Programar partido interno	PartidoInterno, ClubDeportivo.registrarPartido	22, 25	Completo
9 Resultado y estadísticas	Partido.registrarResultado, finalizar y agregarEstadistica; EstadisticaJugador y sus subclases	24, 41, 42	Completo
10 Inscribir equipo en competencia	Competencia.inscribirEquipo, InscripcionCompetencia	19, 20, 25	Completo

Requisito	Clases y métodos principales	Menú	Estado
11 Partido oficial	PartidoOficial, Competencia.programarPartido y actualizarTabla	21, 25	Completo
12 Compra en la tienda	ClubDeportivo.procesarVenta, VentaTienda, DetalleVenta, PoliticaDescuento	5, 7	Completo
13 Compra en la cafetería	VentaCafeteria (impuesto al consumo y propina)	Ninguno	Parcial: la clase y procesarVenta funcionan, pero la consola solo crea VentaTienda.
14 Redimir puntos	Socio.redimirPuntos	Ninguno	Parcial: el socio acumula puntos al comprar y el método existe, pero ninguna opción lo usa.
15 Sugerir un producto	SugerenciaProducto, ClubDeportivo.registrarSugerencia, aprobarSugerencia y rechazarSugerencia	38, 39, 40	Completo
16 Gestionar inventario	ClubDeportivo.reabastecerProducto y trasladarProducto, MovimientoInventario	3, 4, 11	Parcial: el traslado existe, pero no tiene opción en el menú.
17 Solicitar cambio de turno	ClubDeportivo.solicitarCambioTurno, Empleado, Turno	34, 35, 37	Completo
18 Decidir cambio de turno	ClubDeportivo.aprobarCambioTurno, rechazarCambioTurno y validarCobertura; Administrador.aprobarSolicitud	36, 37	Completo

3.3 Reglas del negocio que sí se cumplen

- Una reserva no puede estar en el pasado ni cruzarse con otra de la misma instalación (Reserva.esFechaValida, Instalacion.estaDisponible).

- El número de jugadores debe ajustarse a la capacidad y a la modalidad (Reserva.cantidadJugadoresValida).
- Hay un máximo de instalaciones simultáneas por deporte individual (DeporteIndividual.permiteNuevaReserva).
- Un socio solo puede estar en un equipo activo por deporte y categoría, y el equipo debe tener cupo (inscribirSocioEnEquipo, Equipo.tieneCupo).
- En un partido interno los equipos deben ser distintos, estar activos y ser del mismo deporte y categoría (PartidoInterno).
- En un partido de práctica solo juegan socios con inscripción activa o entrenadores, con máximo 2 participantes en sencillos y 4 en dobles (PartidoPractica).
- Las estadísticas solo se registran cuando el partido está finalizado y para jugadores que participaron (Partido.agregarEstadistica).
- En los partidos oficiales, si la inscripción tiene nómina, solo juegan los de la nómina. El marcador se escribe como “3-1” y la tabla de posiciones se actualiza cuando el partido se finaliza.
- Un equipo solo se inscribe en una competencia si está activo, es del mismo deporte y tiene el mínimo de jugadores (Competencia.inscribirEquipo).
- La asistencia y la evaluación solo se registran para socios habilitados en la sesión, y la nota va de 1 a 5 solo para quien asistió o llegó tarde (RegistroAsistencia).
- El descuento es 15 % para empleados o 8 % con código compartido, y no se acumulan porque cada venta tiene una sola política. IVA de 19 % en tienda, impuesto al consumo de 8 % y propina sugerida de 10 % en cafetería. Los puntos son el 2 % del total.
- No se vende más de lo que hay (InventarioInsuficienteException) y cada cambio de inventario deja un MovimientoInventario.
- Un cambio de turno solo se aprueba si queda la cobertura mínima de entrenadores y fisioterapeutas (ClubDeportivo.validarCobertura).

3.4 Lo que no quedó hecho o tiene limitaciones

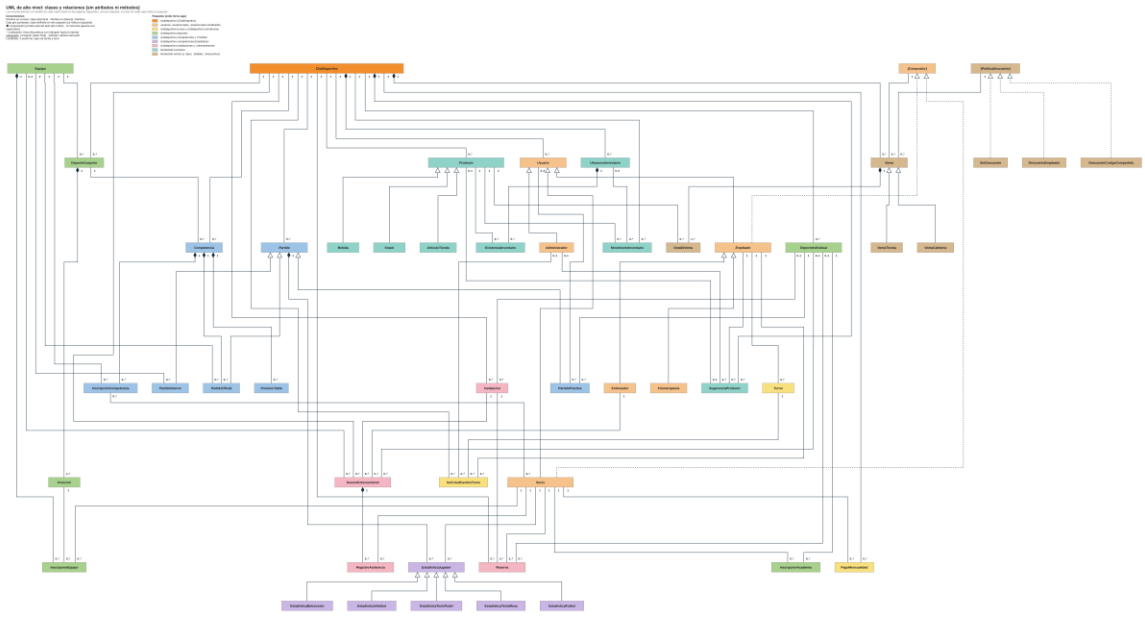
Preferimos decirlo claro para que el documento coincida con el código.

- **Inicio de sesión y menús por rol:** no existen. La consola pregunta el ID para las decisiones del administrador y revisa que sea un Administrador, pero eso no es un login.
- **Categoría y edad:** la fecha de nacimiento se guarda, pero no se compara con la categoría del equipo.
- **Nivel y ranking:** en la práctica no se valida que los niveles sean iguales o adyacentes, y el puntaje de ranking solo se asigna (no hay cálculo ELO).
- **Tarjetas:** las tarjetas se cuentan en la estadística de fútbol, pero no suspenden automáticamente al jugador.
- **ASCUN y Torneo Cerros:** el UML marca el formato como derivado del tipo, pero en el código el formato es un atributo que se guarda. La consola pide los dos datos por separado y el constructor de Competencia no revisa que combinen. No hay fases, grupos ni eliminación directa; solo la tabla de posiciones.
- **Turno del entrenador:** al programar una sesión no se revisa que caiga dentro de su turno (Turno.contieneHorario existe pero no se usa).
- **Bebidas calientes:** ClubDeportivo.validarIngresoProducto existe, pero procesarVenta no la llama, así que la regla de la cancha de pádel y la mesa techada no se aplica.

- **Código compartido:** la consola solo pregunta s/n; no compara con el código de ningún empleado.
- **Ventas guardadas:** en el archivo solo se guarda producto y cantidad, así que al cargar se usa el precio actual del producto.
- **Contraseñas:** se guardan en texto plano en los archivos.
- **Reportes del administrador:** no se implementaron.
- **Pruebas automáticas:** las dejamos para la siguiente etapa.

4. UML de alto nivel

Este diagrama muestra todas las clases del modelo y cómo se relacionan, sin atributos ni métodos. Cada color es un paquete. Los detalles de cada clase están en la sección 5.



(recursos añadidos Tambien dentro del repositorio)

Cómo leerlo:

- Las líneas no tienen verbos. Solo muestran cardinalidad en los extremos. El significado de cada relación se ve en el atributo de la clase (por ejemplo, Reserva tiene un atributo socio y otro instalacion).
- El rombo relleno es composición: el todo está del lado del rombo y las partes no tienen sentido solas (por ejemplo, Venta y DetalleVenta).
- El triángulo vacío es herencia. La línea punteada con triángulo es realización de una interfaz (Comprador y PoliticaDescuento).
- Los nombres en cursiva son clases abstractas (Usuario, Empleado, Partido, EstadisticaJugador, Producto y Venta). Los nombres entre llaves, como {Comprador}, son interfaces.
- ClubDeportivo es el centro del modelo: guarda las listas de usuarios, deportes, instalaciones, reservas, ventas, productos y demás. Por eso tiene tantas líneas.

Los paquetes son estos:

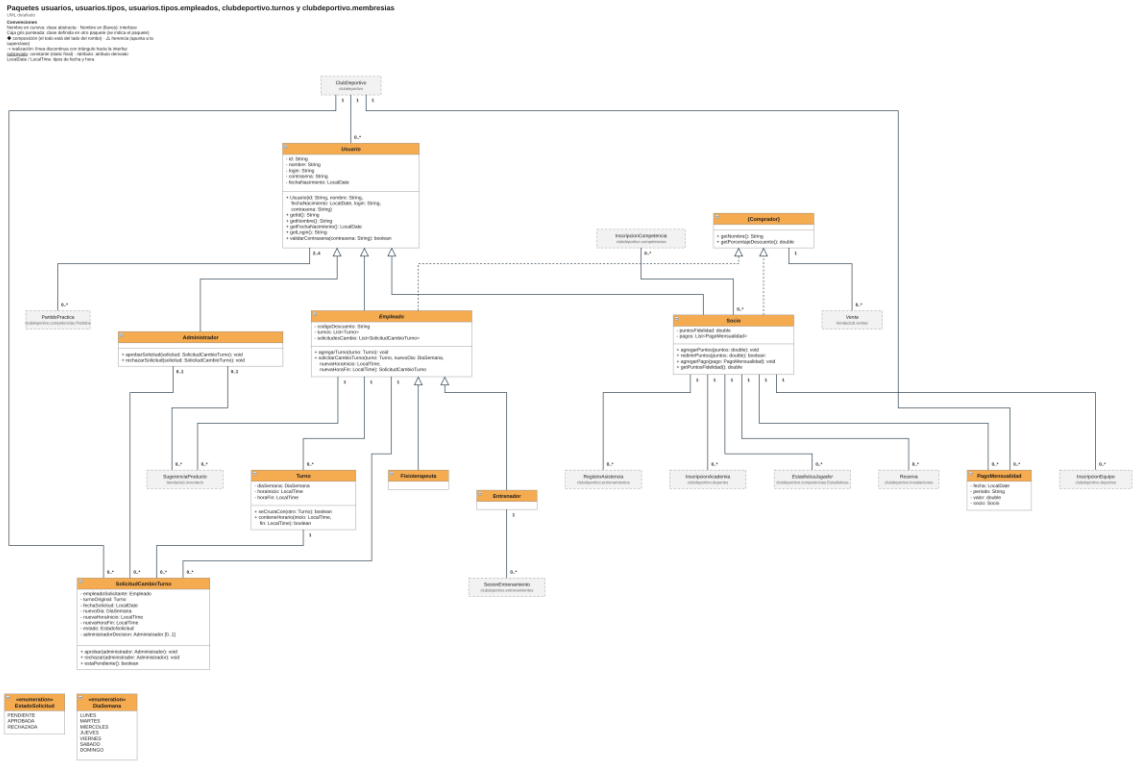
Paquete	Clases principales	Para qué sirve
clubdeportivo	ClubDeportivo	Coordina todo el sistema.
usuarios, usuarios.tipos y usuarios.tipos.empleados	Usuario, Socio, Empleado, Entrenador, Fisioterapeuta, Administrador, Comprador	Las personas del club y quién puede comprar.
clubdeportivo.turnos y clubdeportivo.membresias	Turno, SolicitudCambioTurno, PagoMensualidad	Turnos del personal y pagos de los socios.
clubdeportivo.deportes	DeporteConjunto, DeporteIndividual, Posicion, Equipo, InscripcionEquipo, InscripcionAcademia	Deportes, equipos y academias.
clubdeportivo.competencias (con Partidos y Estadisticas)	Competencia, InscripcionCompetencia, PosicionTabla, Partido y sus tres tipos, EstadisticaJugador y las cinco de cada deporte	Competencias, partidos y estadísticas.
clubdeportivo.instalaciones y clubdeportivo.entrenamientos	Instalacion, Reserva, SesionEntrenamiento, RegistroAsistencia	Reservas y entrenamientos.
tiendaclub.inventario	Producto, ArticuloTienda, Bebida, Snack, UbicacionInventario, ExistenciaInventario, MovimientoInventario, SugerenciaProducto	Productos, existencias, movimientos y sugerencias.
tiendaclub.ventas	Venta, VentaTienda, VentaCafeteria, DetalleVenta, PoliticaDescuento, SinDescuento, DescuentoEmpleado, DescuentoCodigoCompartido	Ventas y descuentos.

Además del UML, el código tiene el paquete utilidades con las excepciones propias (InventarioInsuficienteException y ReservaNoDisponibleException) y la clase PersistenciaClub, que guarda y carga los archivos. Esas clases no son del dominio, por eso no están en el diagrama.

5. UML detallado

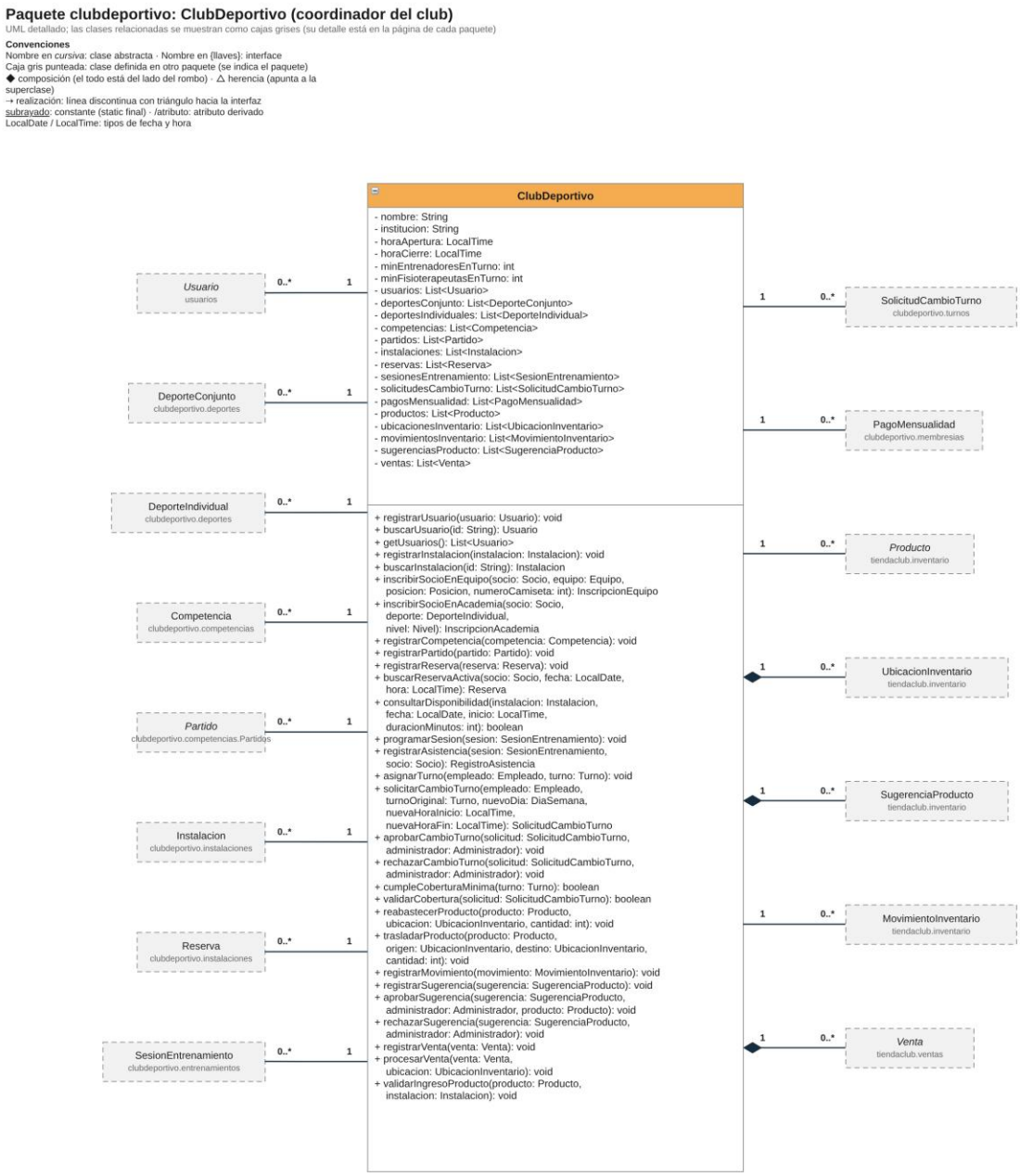
El UML detallado está dividido por paquetes para que se pueda leer. Cada diagrama muestra atributos y métodos de sus clases; las clases que viven en otro paquete aparecen como cajas grises punteadas. Las convenciones son las del diagrama de alto nivel. Además, las constantes (static final) van subrayadas y los atributos derivados llevan una barra (/).

5.1 Usuarios y turnos



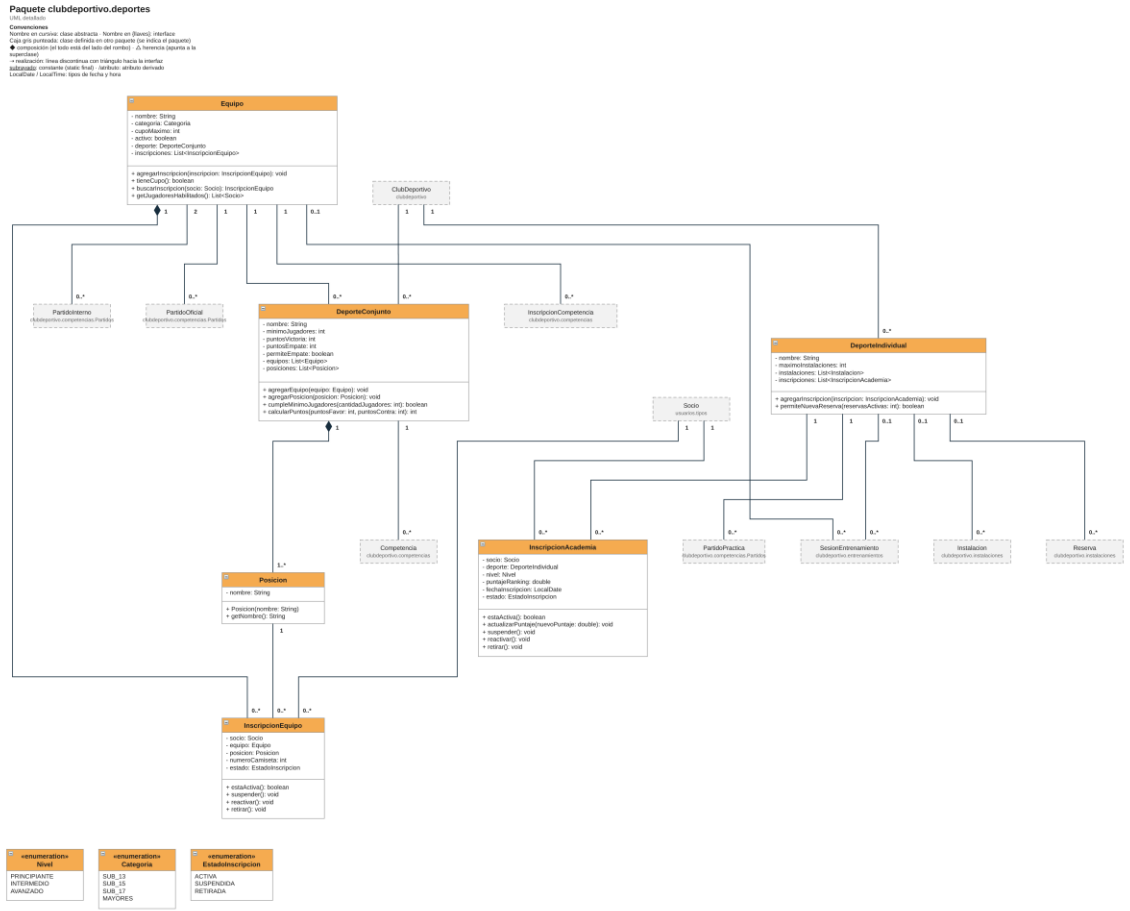
- Usuario es abstracta y concentra id, nombre, fecha de nacimiento, login y contraseña. De ella heredan Socio, Empleado y Administrador.
- Empleado también es abstracta; sus especializaciones son Entrenador y Fisioterapeuta.
- Socio y Empleado implementan la interfaz Comprador.
- Administrador hereda de Usuario y no de Empleado, porque no compra con descuento de empleado, no tiene turnos propios y no juega.
- SolicitudCambioTurno guarda el turno original, el nuevo horario, el estado y el administrador que decidió (0..1, queda vacío mientras está pendiente).

5.2 ClubDeportivo



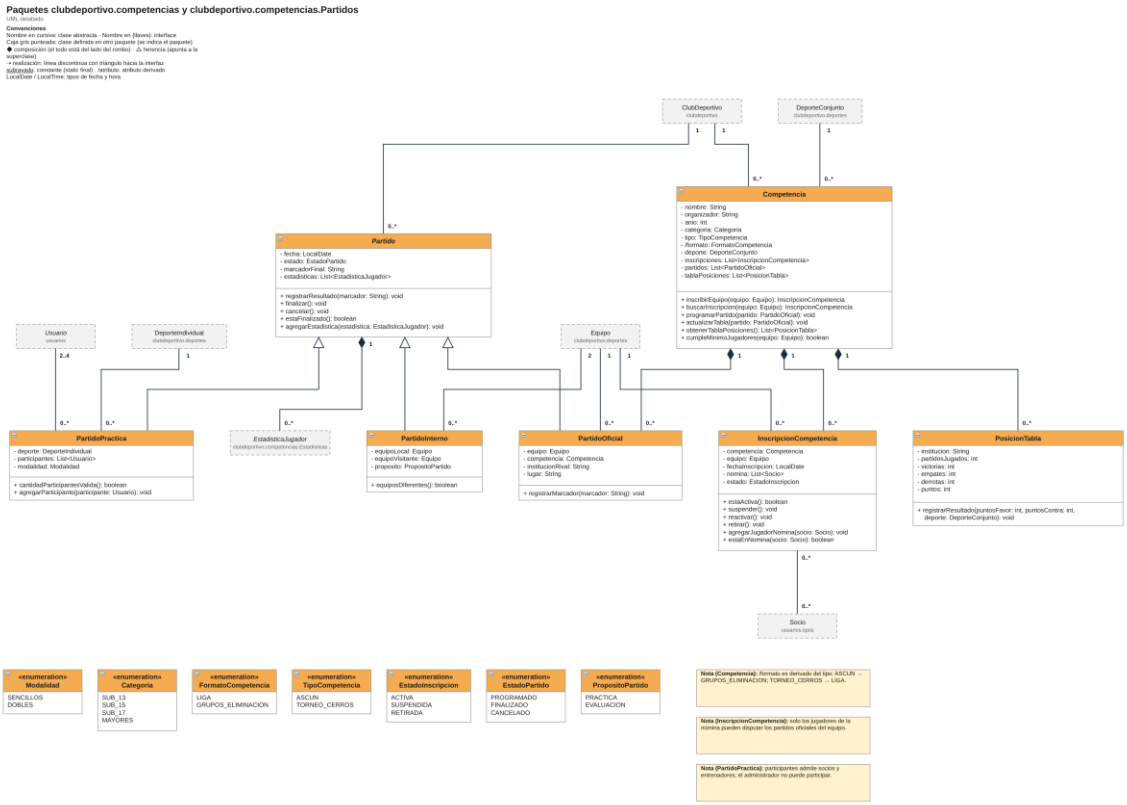
- ClubDeportivo guarda las listas del club (usuarios, deportes, competencias, partidos, instalaciones, reservas, sesiones, solicitudes de turno, pagos, productos, ubicaciones, movimientos, sugerencias y ventas) y los datos del club: horario de apertura y cierre, y la cobertura mínima de entrenadores y fisioterapeutas.
- Sus operaciones se agrupan por tema: usuarios, instalaciones y reservas, inscripciones, competencias y partidos, entrenamiento, turnos, inventario, sugerencias y ventas.

5.3 Deportes



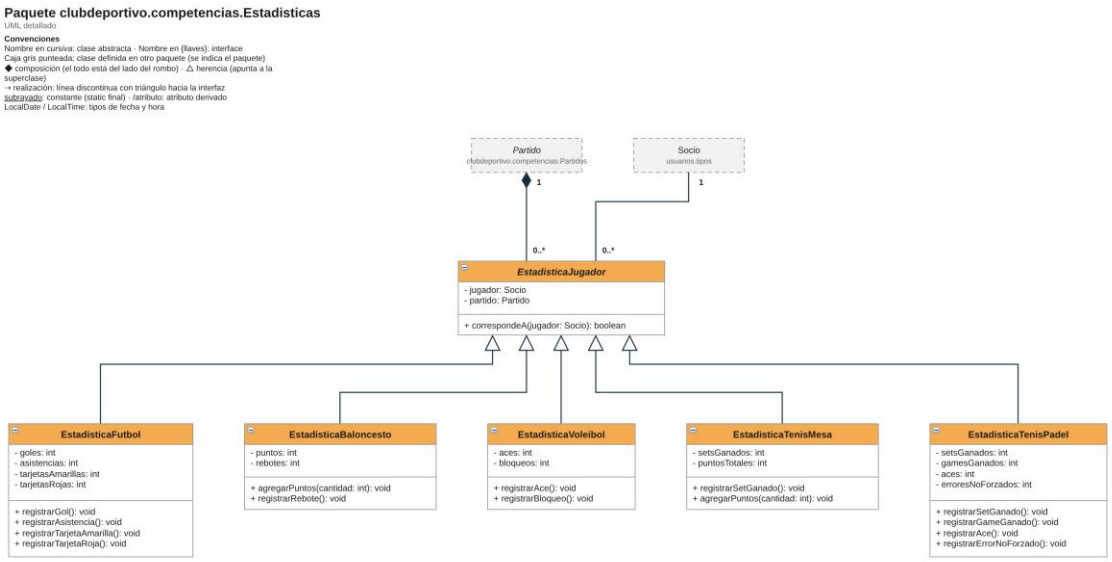
- DeporteConjunto tiene sus posiciones (composición con Posicion), sus equipos, el mínimo de jugadores y los puntos por victoria y empate.
- Equipo tiene categoría, cupo máximo y si está activo. Un socio entra a un equipo por medio de InscripcionEquipo, que guarda su posición, su número de camiseta y su estado.
- DeporteIndividual guarda el máximo de instalaciones simultáneas y sus inscripciones. InscripcionAcademia guarda el nivel y el puntaje de ranking del socio en ese deporte.

5.4 Competencias y partidos



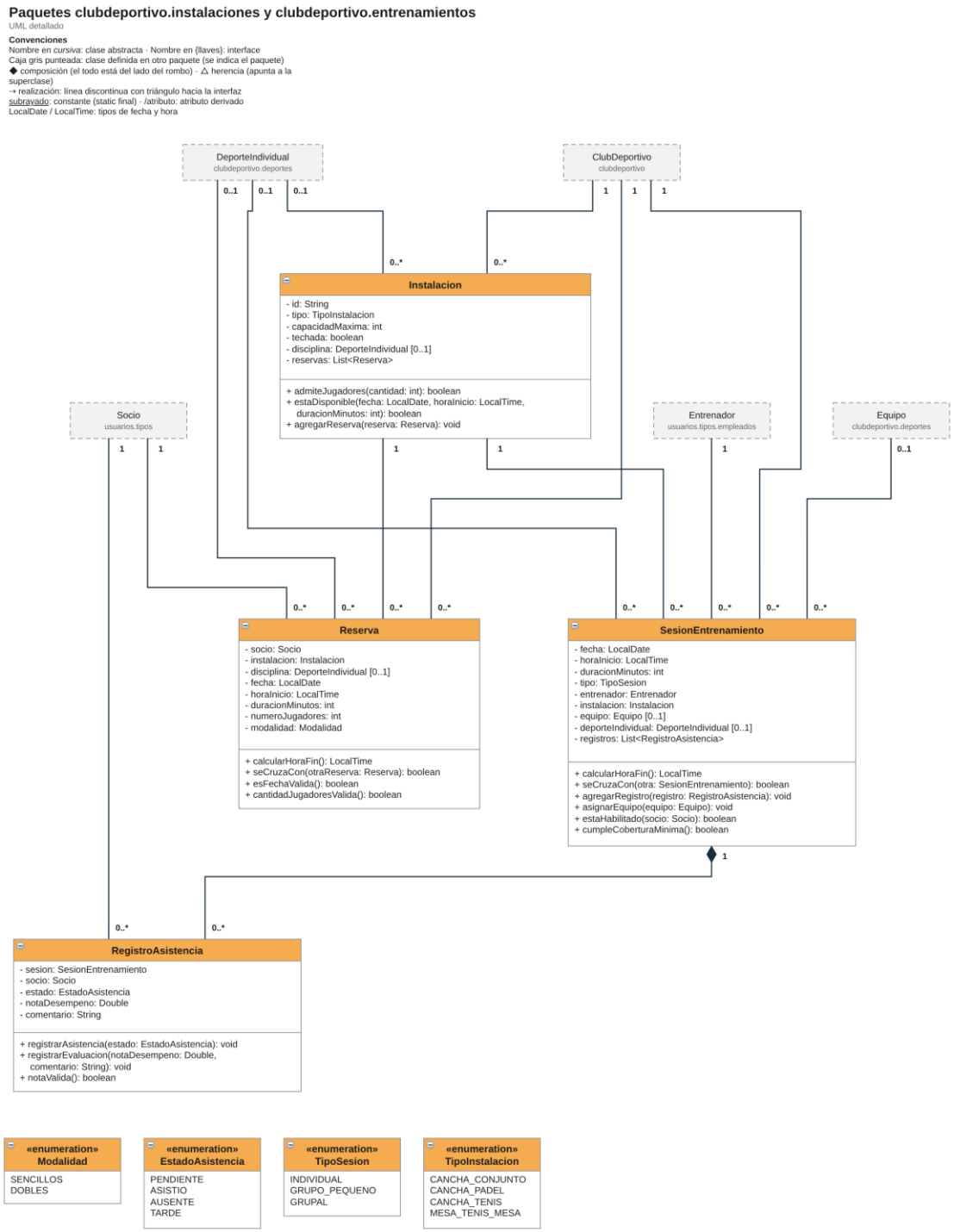
- Partido es abstracta con tres tipos: PartidoInterno (dos equipos del club), PartidoPractica (de 2 a 4 participantes en un deporte individual) y PartidoOficial (un equipo, una competencia y una institución rival).
- Competencia tiene sus inscripciones, sus partidos oficiales y la tabla de posiciones (lista de PosicionTabla). InscripcionCompetencia guarda la nómina de jugadores del equipo.

5.5 Estadísticas



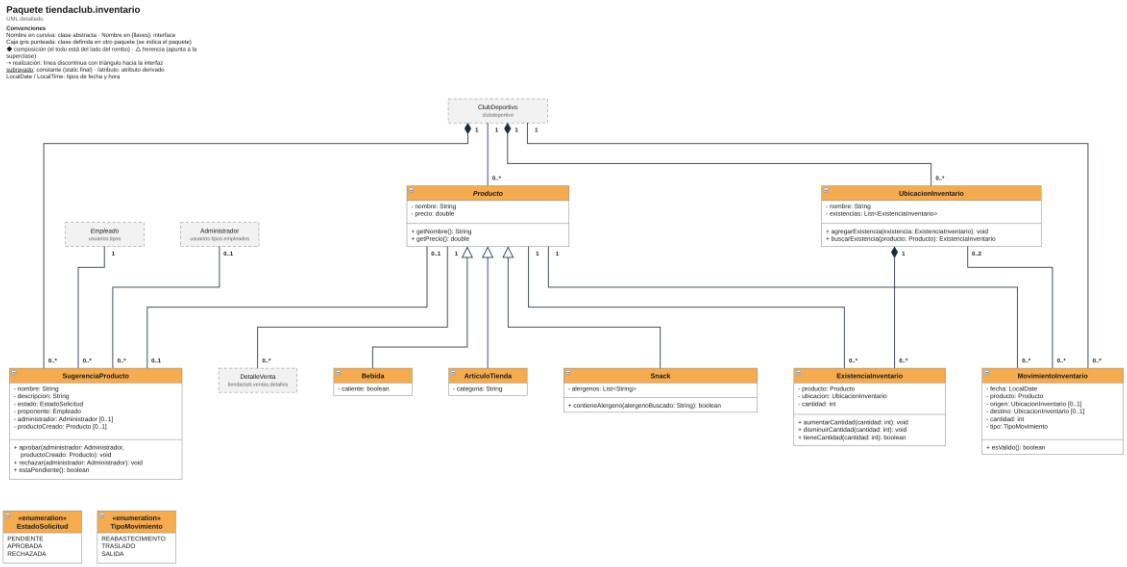
- EstadisticaJugador es abstracta: guarda el jugador y el partido. Hay una subclase por deporte (fútbol, baloncesto, voleibol, tenis de mesa y tenis pádel) porque cada deporte cuenta cosas distintas.

5.6 Instalaciones y entrenamientos



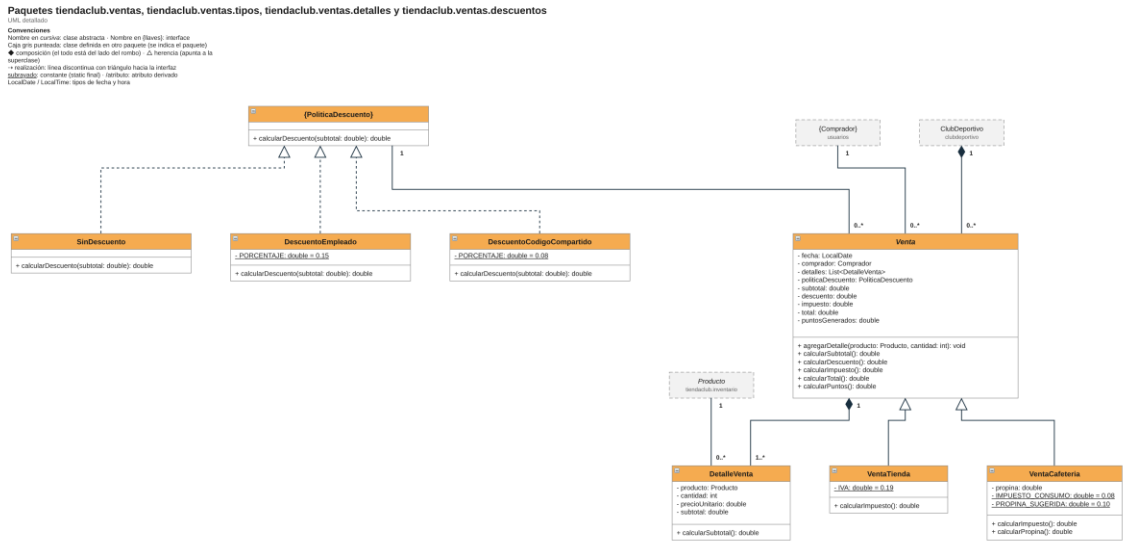
- Instalacion tiene su tipo (TipoInstalacion), su capacidad, si es techada y su lista de reservas. Reserva calcula su hora final y revisa si se cruza con otra.
- SesionEntrenamiento guarda su entrenador, su instalación y su equipo o su deporte individual. Cada socio de la sesión tiene un RegistroAsistencia con su estado, su nota y un comentario.

5.7 Tienda: inventario



- Producto es abstracta, con tres tipos: ArticuloTienda (categoría), Bebida (si es caliente) y Snack (lista de alérgenos).
- UbicacionInventario tiene sus existencias (composición). ExistenciaInventario es la cantidad de un producto en una ubicación.
- MovimientoInventario registra fecha, producto, cantidad, tipo y, de forma opcional, el origen y el destino.
- SugerenciaProducto guarda quién la propuso, su estado y, si se aprueba, el administrador y el producto creado.

5.8 Tienda: ventas



- Venta es abstracta: guarda el comprador, sus detalles (composición con DetalleVenta) y la política de descuento, y calcula subtotal, descuento, impuesto, total y puntos.
- VentaTienda calcula el IVA y VentaCafeteria calcula el impuesto al consumo y la propina.
- PoliticaDescuento es una interfaz con tres implementaciones: SinDescuento, DescuentoEmpleado y DescuentoCodigoCompartido.

5.9 Enumeraciones

Enumeración	Valores
Categoria	SUB_13, SUB_15, SUB_17, MAYORES
Nivel	PRINCIPIANTE, INTERMEDIO, AVANZADO
Modalidad	SENCILLOS, DOBLES
EstadoInscripcion	ACTIVA, SUSPENDIDA, RETIRADA
TipoCompetencia	ASCUN, TORNEO_CERROS
FormatoCompetencia	LIGA, GRUPOS_ELIMINACION
EstadoPartido	PROGRAMADO, FINALIZADO, CANCELADO
PropositoPartido	PRACTICA, EVALUACION
TipoInstalacion	CANCHA_CONJUNTO, CANCHA_PADEL, CANCHA_TENIS, MESA_TENIS_MESA
TipoSesion	INDIVIDUAL, GRUPO_PEQUENO, GRUPAL
EstadoAsistencia	PENDIENTE, ASISTIO, AUSENTE, TARDE
DiaSemana	LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO
EstadoSolicitud	PENDIENTE, APROBADA, RECHAZADA
TipoMovimiento	REABASTECIMIENTO, TRASLADO, SALIDA

Los archivos de origen de los diagramas (.drawio) van junto con este documento en la entrega.

6. Asignación de responsabilidades

La idea fue que cada clase se encargue de lo que conoce y que ClubDeportivo coordine cuando una regla involucra varios objetos. Así, por ejemplo, Reserva calcula su hora final, Venta calcula sus totales y las políticas calculan los descuentos, mientras que ClubDeportivo revisa que la instalación exista, que haya disponibilidad y que se cumpla el máximo de la disciplina.

6.1 Personas, turnos y pagos

Clase	Responsabilidad	Colabora con
Usuario	Guarda id, nombre, fecha de nacimiento, login y contraseña, y valida la contraseña.	Socio, Empleado, Administrador
Socio	Acumula y redime puntos de fidelidad y lleva sus pagos. Es Comprador sin descuento.	PagoMensualidad, Venta
Empleado	Guarda sus turnos, sus solicitudes de cambio y su código de descuento. Es Comprador con 15 % de descuento.	Turno, SolicitudCambioTurno
Entrenador y Fisioterapeuta	Son los dos tipos de personal que cuentan para la cobertura de turnos. El entrenador dirige las sesiones.	SesionEntrenamiento, ClubDeportivo
Administrador	Aprueba o rechaza solicitudes (de turno y de producto).	SolicitudCambioTurno, SugerenciaProducto
Comprador (interfaz)	Define lo mínimo que necesita una venta de quien compra: su nombre y su porcentaje de descuento.	Venta
Turno	Conoce su día y sus horas, y sabe si se cruza con otro turno o si contiene un horario.	Empleado
SolicitudCambioTurno	Guarda el cambio pedido y su estado. Sabe aprobarse, rechazarse y decir si sigue pendiente.	Empleado, Administrador
PagoMensualidad	Guarda la fecha, el periodo, el valor y el socio del pago.	Socio

6.2 Coordinación del club

Clase	Responsabilidad	Colabora con
ClubDeportivo	Guarda las listas del club y busca elementos en ellas. Coordina las operaciones que involucran varios objetos: reservar, inscribir, programar sesiones, procesar ventas, aprobar cambios de turno (cobertura mínima), mover inventario y decidir sugerencias.	Casi todas las clases

6.3 Deportes, competencias y partidos

Clase	Responsabilidad	Colabora con
DeporteConjunto	Guarda sus equipos y posiciones, el mínimo de jugadores y los puntos por victoria y empate. Calcula los puntos de un resultado.	Equipo, Posicion, Competencia
DeporteIndividual	Guarda sus inscripciones e instalaciones. Dice si se permite una nueva reserva según el máximo de instalaciones simultáneas.	InscripcionAcademia, Instalacion
Equipo	Guarda sus inscripciones, revisa el cupo y devuelve los jugadores habilitados.	InscripcionEquipo, DeporteConjunto
InscripcionEquipo	Representa a un socio en un equipo: posición, camiseta y estado (activa, suspendida o retirada).	Socio, Equipo, Posicion
InscripcionAcademia	Representa a un socio en un deporte individual: nivel, puntaje de ranking y estado.	Socio, DeporteIndividual
Competencia	Inscribe equipos (revisa que estén activos, sean del deporte y tengan el mínimo de jugadores), programa partidos oficiales, valida el marcador y actualiza la tabla de posiciones.	InscripcionCompetencia, PartidoOficial, PosicionTabla
InscripcionCompetencia	Guarda la participación de un equipo: fecha, estado y nómina de jugadores.	Competencia, Equipo, Socio
PosicionTabla	Acumula partidos jugados, victorias, empates, derrotas y puntos de una institución.	Competencia, DeporteConjunto
Partido (y sus tres tipos)	Guarda fecha, estado y marcador, y se puede finalizar o cancelar. Solo acepta estadísticas si está finalizado y el jugador participó. PartidoInterno revisa sus equipos, PartidoPractica revisa participantes y modalidad, y PartidoOficial revisa el marcador y actualiza la tabla al finalizar.	Equipo, Competencia, EstadisticaJugador

Clase	Responsabilidad	Colabora con
EstadisticaJugador (y sus cinco subclases)	Guarda lo que hizo un jugador en un partido, según el deporte.	Socio, Partido

6.4 Instalaciones y entrenamiento

Clase	Responsabilidad	Colabora con
Instalacion	Guarda su tipo, su capacidad y sus reservas. Dice si admite cierta cantidad de jugadores y si está disponible en un horario.	Reserva, DeporteIndividual
Reserva	Calcula su hora final, revisa si se cruza con otra y si su fecha y su número de jugadores son válidos.	Socio, Instalacion
SesionEntrenamiento	Calcula su hora final, revisa cruces, guarda los registros de asistencia y dice si un socio está habilitado para la sesión.	Entrenador, Instalacion, Equipo, RegistroAsistencia
RegistroAsistencia	Guarda el estado de asistencia, la nota (1 a 5) y el comentario de un socio en una sesión.	SesionEntrenamiento, Socio

6.5 Tienda e inventario

Clase	Responsabilidad	Colabora con
Producto (ArticuloTienda, Bebida y Snack)	Guarda nombre y precio, más el dato propio de cada tipo (categoría, si es caliente o los alérgenos).	ExistenciaInventario, DetalleVenta
UbicacionInventario	Guarda las existencias de un lugar y busca la de un producto.	ExistenciaInventario
ExistenciaInventario	Guarda la cantidad de un producto en una ubicación. Aumenta, disminuye y dice si hay suficiente.	Producto, UbicacionInventario
MovimientoInventario	Registra un cambio de inventario (reabastecimiento, traslado o salida) y revisa que sus datos sean válidos.	Producto, UbicacionInventario
SugerenciaProducto	Guarda la propuesta de un empleado y su estado. Se aprueba (con el producto creado) o se rechaza.	Empleado, Administrador

Clase	Responsabilidad	Colabora con
Venta	Guarda comprador, detalles y política de descuento. Calcula subtotal, descuento (se lo pide a la política), impuesto, total y puntos.	Comprador, DetalleVenta, PoliticaDescuento
VentaTienda y VentaCafeteria	Calculan su impuesto: IVA en la tienda; impuesto al consumo y propina en la cafetería.	Venta
DetalleVenta	Guarda producto, cantidad, precio unitario y subtotal de una línea de la venta.	Venta, Producto
PoliticaDescuento y sus tres implementaciones	Cada una calcula el descuento sobre un subtotal.	Venta

6.6 Clases de apoyo (fuera del UML)

Clase	Responsabilidad
PersistenciaClub	Guarda y carga los 23 archivos de la carpeta datos (sección 9).
InventarioInsuficienteException y ReservaNoDisponibleException	Avisan los dos errores de negocio más importantes: vender más de lo que hay y reservar algo no disponible.
ConsolaClubDeportivo	Pide datos, llama a ClubDeportivo y muestra los resultados. No tiene reglas del negocio.

Una observación honesta: ClubDeportivo es la clase más grande del proyecto y tiene varias validaciones propias. Dejamos ahí las que necesitan mirar varias clases a la vez, pero algunas podrían moverse a las clases del dominio en una mejora futura.

7. Decisiones de diseño y justificaciones

Para cada decisión contamos qué hicimos, por qué y qué nos costó. Usamos los principios SOLID de la guía como criterio.

7.1 Descuentos con el patrón Strategy

- **Qué hicimos:** la interfaz PoliticaDescuento tiene un método, calcularDescuento(subtotal), y tres clases la implementan: SinDescuento (0), DescuentoEmpleado (15 %) y DescuentoCodigoCompartido (8 %). Venta guarda una politicaDescuento y le pregunta el descuento; no conoce las fórmulas.
- **Por qué:** en la Entrega 1 Venta tenía un enum tipoDescuento, y eso obliga a llenar la clase de if/else. Con Strategy, agregar un descuento nuevo es crear una clase y Venta no cambia (principio abierto/cerrado). Además Venta depende de la interfaz y no de las clases concretas

(inversión de dependencias). Como cada venta tiene una sola política, los descuentos no se acumulan, que es lo que pide el enunciado.

- **Costo y limitación:** son tres clases muy pequeñas. Además, la política la escoge finalizarVenta (en VentaTienda y VentaCafeteria) con un if/else según el código compartido o el descuento del comprador, y Venta usa SinDescuento como política por defecto. La fórmula quedó desacoplada, pero la elección todavía no.

7.2 Interfaces pequeñas: Comprador y PoliticaDescuento

- **Qué hicimos:** Comprador solo pide dos cosas (getNombre y getPorcentajeDescuento) y PoliticaDescuento pide una. Socio y Empleado implementan Comprador; Administrador no.
- **Por qué:** es el principio de segregación de interfaces: nadie queda obligado a implementar métodos que no usa. Venta no necesita saber si compra un socio o un empleado, y el administrador no puede comprar porque no es Comprador.
- **Costo:** procesarVenta todavía pregunta si el comprador es un Socio (instanceof) para sumarle los puntos, porque solo el socio tiene puntos de fidelidad.

7.3 Herencia solo donde de verdad hay especialización

- **Qué hicimos:** usamos herencia en usuarios (Usuario, con Socio, Empleado y Administrador; y Empleado con Entrenador y Fisioterapeuta), productos, partidos, estadísticas y ventas.
- **Por qué:** es el principio de sustitución de Liskov: donde se espera el padre sirve cualquier hijo. Por ejemplo, procesarVenta recibe cualquier Venta y registrarPartido recibe cualquier Partido. Administrador hereda de Usuario y no de Empleado porque, si fuera Empleado, tendría turnos, descuento de empleado y podría comprar, cosas que el enunciado no le da.
- **Costo:** más clases (por ejemplo, una subclase de estadística por deporte), pero evitamos una sola clase con muchos atributos que no aplican a todos los deportes.

7.4 Una sola clase Instalacion con TipoInstalacion

- **Qué hicimos:** en la Entrega 1 había cuatro subclases (CanchaConjunto, CanchaPadel, CanchaTenis y MesaTenisMesa). Las reemplazamos por Instalacion con el atributo tipo.
- **Por qué:** las subclases solo cambiaban de nombre y no tenían comportamiento propio, así que la herencia no aportaba nada.
- **Costo:** las reglas por tipo (cuántos jugadores acepta cada una) quedaron en un switch dentro de Reserva.cantidadJugadoresValida. Si apareciera un tipo nuevo habría que modificar ese método.

7.5 Enumeraciones para los valores cerrados

- **Qué hicimos:** Categoria, Nivel, Modalidad, TipoInstalacion, los estados y los tipos son enums (sección 5.9).
- **Por qué:** el compilador revisa los valores y no se escriben textos con errores.
- **Costo:** los valores no se pueden configurar mientras corre el programa. Por ejemplo, las edades de una categoría no son configurables.

7.6 Inscripciones como clases con estado

- **Qué hicimos:** InscripcionEquipo, InscripcionAcademia e InscripcionCompetencia son clases con su propio estado (ACTIVA, SUSPENDIDA o RETIRADA).
- **Por qué:** la relación tiene datos propios (posición, camiseta, nivel, ranking, nómina, fecha). Y como el estado está en cada inscripción, un socio puede estar suspendido en un deporte y activo en otro.
- **Costo:** para saber si un socio está habilitado hay que buscar su inscripción (Equipo.buscarInscripcion).

7.7 DeporteConjunto y DeporteIndividual separados

- **Qué hicimos:** no hay una clase Deporte común. ClubDeportivo guarda una lista de cada tipo.
- **Por qué:** los deportes de conjunto manejan equipos, posiciones y puntos; los individuales manejan inscripciones y el máximo de instalaciones. Con una sola clase cada una tendría atributos que no usa.
- **Costo:** el atributo nombre se repite en las dos.

7.8 Composición para las partes que dependen del todo

- **Qué hicimos:** usamos composición (rombo relleno) en Venta con DetalleVenta, UbicacionInventario con ExistenciaInventario, DeporteConjunto con Posicion, Equipo con InscripcionEquipo, Competencia con sus inscripciones, partidos oficiales y tabla de posiciones, SesionEntrenamiento con RegistroAsistencia y Partido con EstadisticaJugador.
- **Por qué:** esas partes no tienen sentido sin el todo; una línea de venta no existe sin su venta.

7.9 ASCUN y Torneo de los Cerros

- **Qué hicimos:** según el enunciado, ASCUN tiene fase de grupos y eliminación directa, y el Torneo de los Cerros es una liga todos contra todos con tabla de posiciones. Los separamos con dos enums: TipoCompetencia (cuál competencia es) y FormatoCompetencia (cómo se organiza). En el UML el formato es un atributo derivado del tipo (ASCUN → GRUPOS_ELIMINACION y TORNEO_CERROS → LIGA), para que no existan combinaciones imposibles.
- **Por qué:** en la Entrega 1 los dos atributos eran independientes y se podía crear un Torneo de los Cerros con grupos. También agregamos PosicionTabla y los puntos por deporte (DeporteConjunto.calcularPuntos) para la tabla.
- **Limitación:** en el código el formato se guarda como atributo y no se valida la combinación con el tipo, y no armamos fases ni eliminación directa; solo la tabla de posiciones. Lo dejamos así porque las fases no hacían parte de las historias HU-01 a HU-18.

7.10 ClubDeportivo como coordinador

- **Qué hicimos:** ClubDeportivo guarda las listas del club y tiene las operaciones que miran varios objetos a la vez.
- **Por qué:** reglas como “no vender más de lo que hay”, “el máximo de instalaciones de la disciplina” o “cobertura mínima de turnos” necesitan ver inventario, reservas o todo el personal. Una sola entidad no puede hacerlo. Las demás clases mantienen sus reglas propias (sección 6).

- **Costo:** ClubDeportivo quedó muy grande (más de 1.300 líneas) y con muchas relaciones. Es lo que más mejoraríamos.

7.11 Relaciones como atributos y colecciones List

- **Qué hicimos:** las líneas del UML no llevan verbos; cada relación es un atributo, y las colecciones se declaran como List.
- **Por qué:** el UML se traduce directo a Java y la guía recomienda programar contra la interfaz List.
- **Costo:** para entender una relación hay que leer los atributos de la clase.

7.12 Lo que mejoraríamos

- Pasar la elección de la política de descuento a quien crea la venta, para que finalizarVenta no tenga if/else.
- Quitar el instanceof de procesarVenta, por ejemplo haciendo que el comprador reciba sus propios puntos.
- Las dos excepciones propias extienden RuntimeException; la guía recomienda que las de negocio sean checked.
- Dividir PersistenciaClub por grupos de archivos: hoy es una sola clase de unas 2.000 líneas.
- Aplicar las reglas pendientes de la sección 3.4.

8. Diagramas de secuencia

Escogimos tres requerimientos críticos porque son los que tienen más reglas y mezclan varias clases: la venta con descuento y puntos (HU-12 y HU-13), la aprobación de un cambio de turno con cobertura mínima (HU-18) y la reserva con el máximo de instalaciones (HU-02). Los diagramas siguen las llamadas reales del código. Las flechas punteadas son respuestas o errores, y lo que va entre corchetes solo pasa si se cumple esa condición.

8.1 Venta con descuento y puntos (procesarVenta)

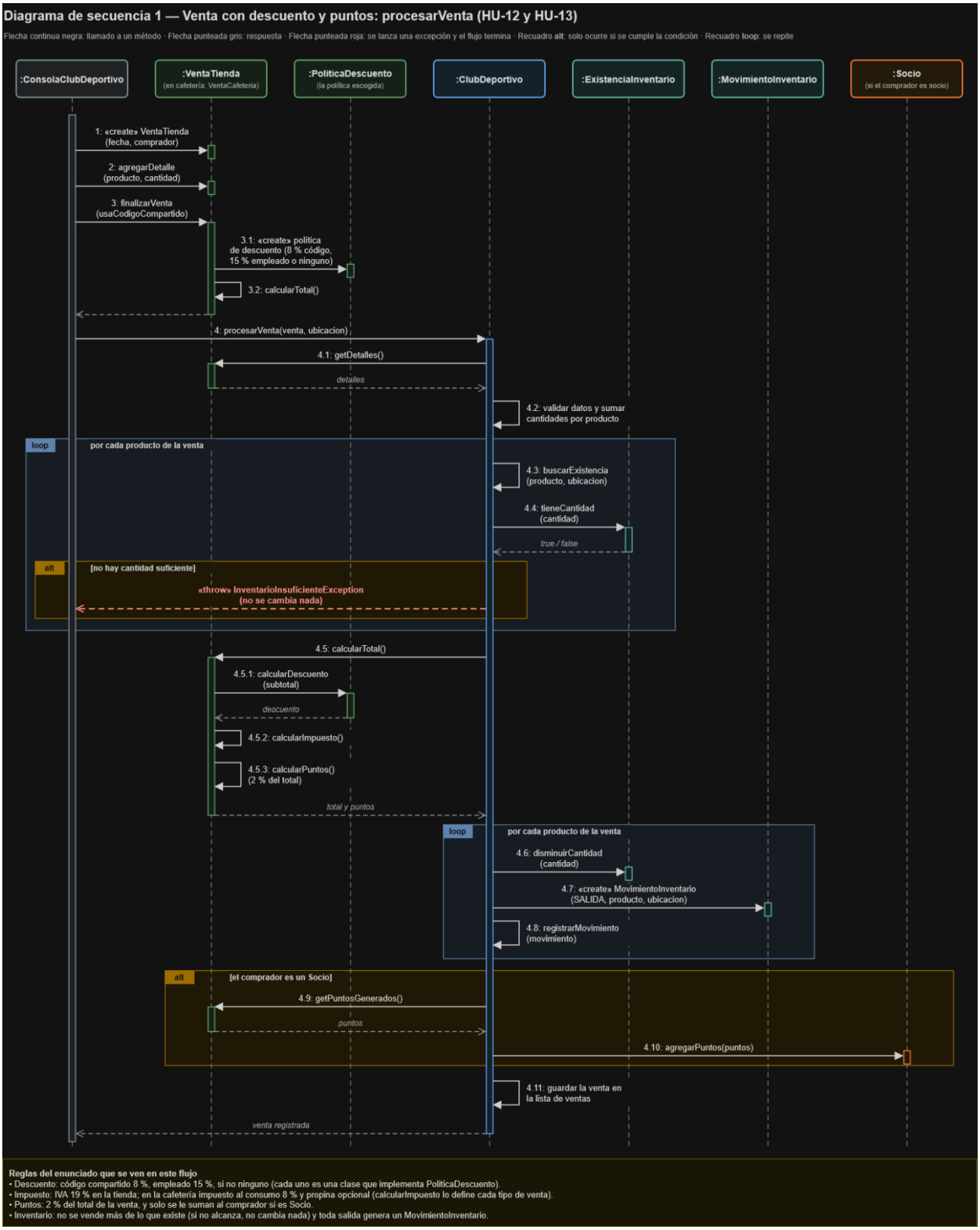


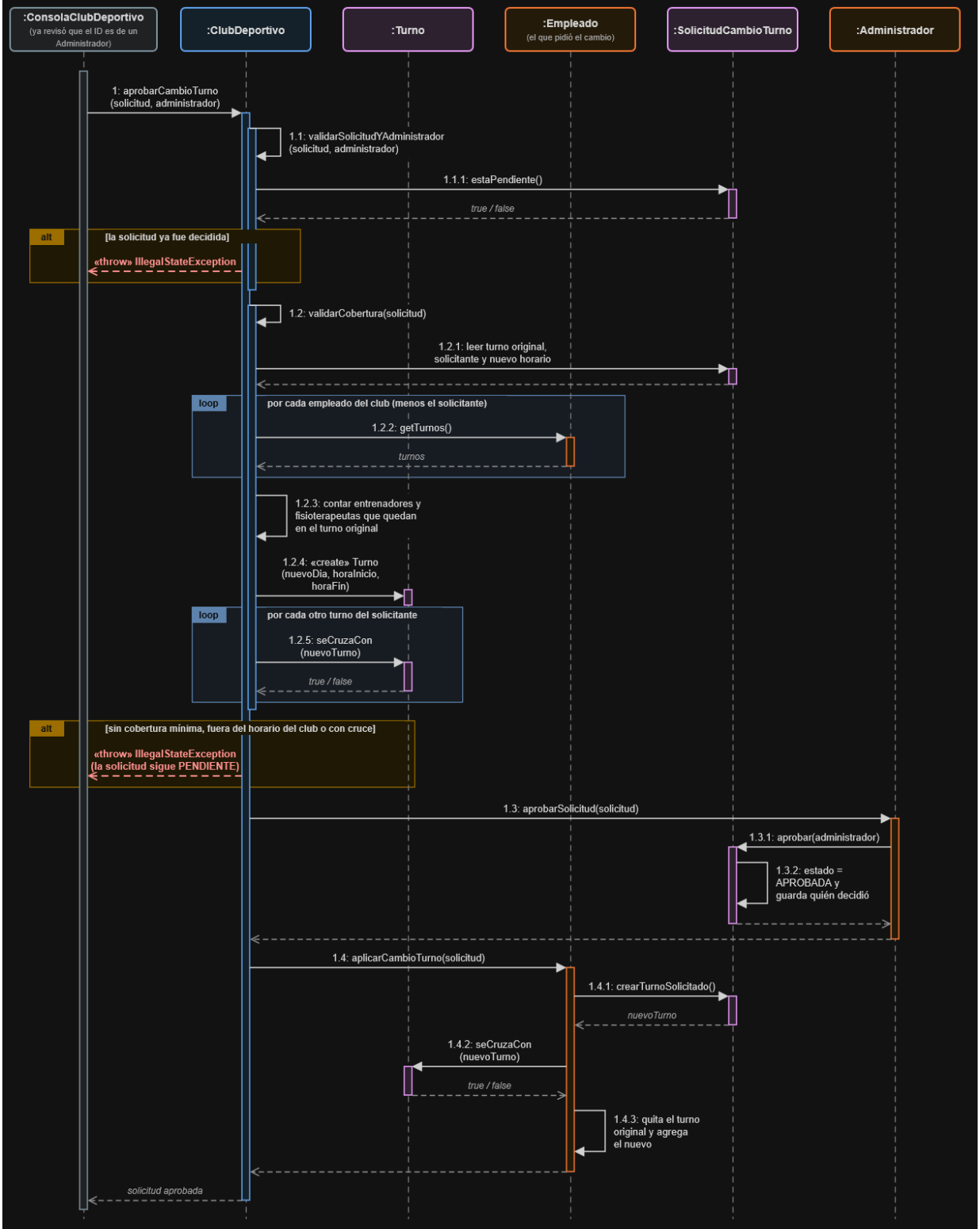
Diagrama de secuencia de procesarVenta (HU-12 y HU-13)

- Antes de procesar, la consola crea la venta (en el menú actual, una VentaTienda), le agrega el detalle y llama a finalizarVenta, que escoge la política de descuento: código compartido (8 %), descuento de empleado (15 %) o ninguno.
- ClubDeportivo suma las cantidades por producto y le pregunta a ExistenciaInventario si alcanza. Si no alcanza, lanza InventarioInsuficienteException y no cambia nada.
- Venta calcula su total: le pide el descuento a la PoliticaDescuento, calcula el impuesto (IVA de 19 % en tienda; impuesto al consumo de 8 % y propina en cafetería) y los puntos (2 % del total).
- Si todo salió bien, ClubDeportivo descuenta el inventario, registra un MovimientoInventario de tipo SALIDA, le suma los puntos al socio (solo si el comprador es Socio) y guarda la venta.

8.2 Cambio de turno con cobertura mínima (aprobarCambioTurno)

Diagrama de secuencia 2 — Cambio de turno con cobertura mínima: aprobarCambioTurno (HU-18)

Flecha continua negra: llamado a un método · Flecha punteada gris: respuesta · Flecha punteada roja: se lanza una excepción y el flujo termina · Recuadro **alt**: solo ocurre si se cumple la condición · Recuadro **loop**: se repite



Reglas del enunciado que se ven en este flujo

- Solo el administrador decide, y solo sobre solicitudes que siguen pendientes.
- Cobertura mínima: al quitar al solicitante del turno original deben quedar al menos los entrenadores y fisioterapeutas mínimos del club.
- El nuevo horario debe estar dentro del horario de apertura y cierre y no cruzarse con otros turnos del mismo empleado.
- Si algo falla se lanza la excepción y la solicitud no cambia de estado.

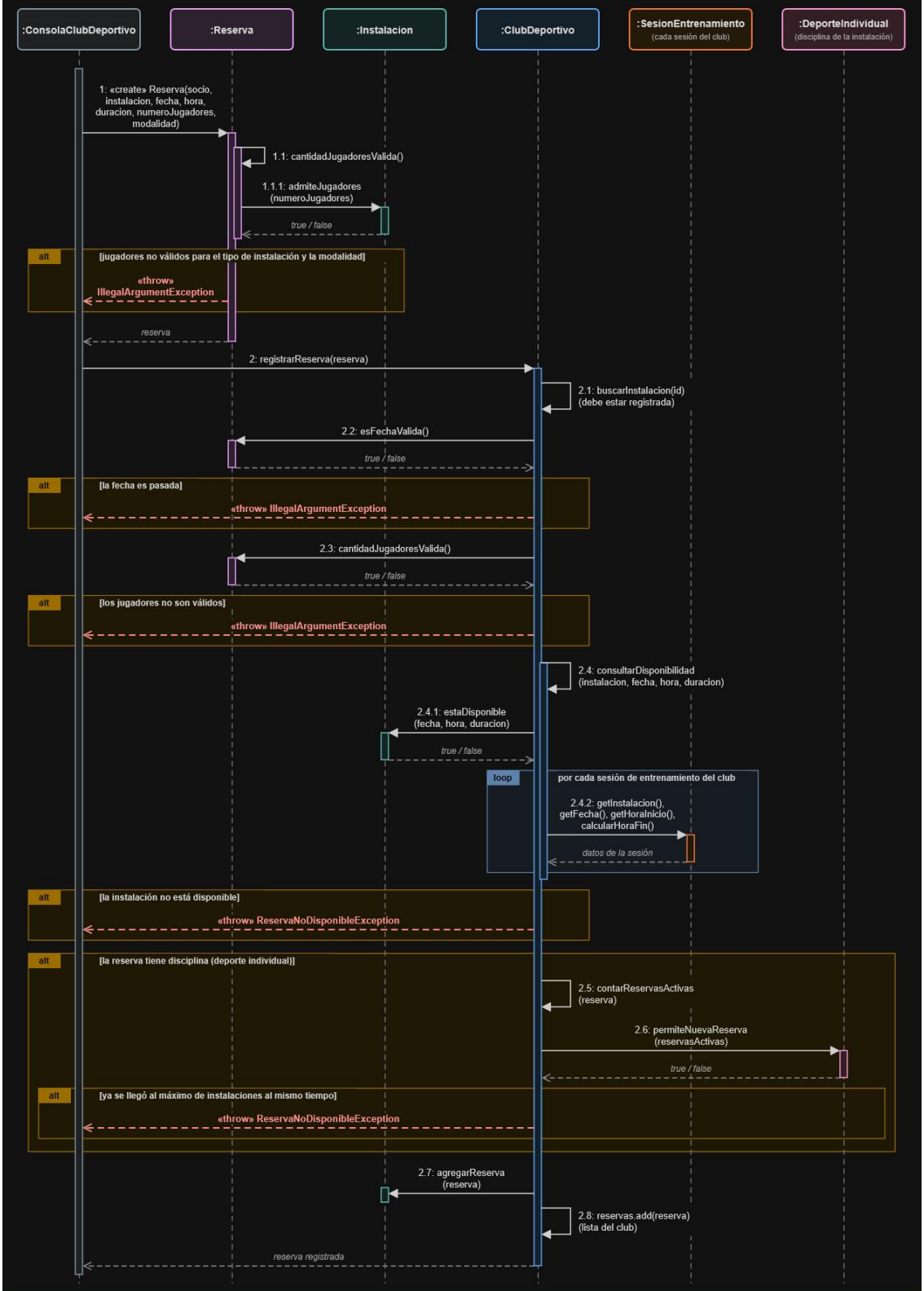
Diagrama de secuencia de aprobarCambioTurno (HU-18)

- Antes de llamar, la consola revisa que el ID sea de un Administrador (todavía no hay inicio de sesión).
- ClubDeportivo revisa que la solicitud y el administrador estén registrados y que la solicitud siga pendiente.
- validarCobertura cuenta cuántos entrenadores y fisioterapeutas se quedan en el turno original sin el empleado que pide el cambio, y los compara con el mínimo del club. También revisa que el nuevo horario esté dentro del horario del club y que no se cruce con otros turnos del mismo empleado. Si algo falla, lanza IllegalStateException y la solicitud sigue pendiente.
- Si pasa, el Administrador aprueba la solicitud (queda en estado APROBADA con el administrador que decidió) y el empleado cambia su turno original por el nuevo.

8.3 Reserva con máximo de instalaciones (registrarReserva)

Diagrama de secuencia 3 — Reserva con máximo de instalaciones: registrarReserva (HU-02)

Flecha continua negra: llamado a un método · Flecha punteada gris: respuesta · Flecha punteada roja: se lanza una excepción y el flujo termina · Recuadro **alt**: solo ocurre si se cumple la condición · Recuadro **loop**: se repite



Reglas del enunciado que se ven en este flujo

- La reserva no puede estar en el pasado ni cruzarse con otra de la misma instalación (ni con una sesión de entrenamiento en esa instalación).
- La cantidad de jugadores debe ajustarse a la capacidad y a la modalidad de la instalación (por ejemplo, tenis: 2 sencillos o 4 dobles).
- Un deporte individual tiene un máximo de instalaciones reservadas al mismo tiempo (DeporteIndividual.permiteNuevaReserva).

Diagrama de secuencia de registrarReserva (HU-02)

- Al crear la Reserva se revisa que el número de jugadores corresponda al tipo de instalación y a la modalidad.
- ClubDeportivo revisa que la instalación esté registrada, que la fecha no sea pasada y que los jugadores sean válidos.
- La disponibilidad se consulta con consultarDisponibilidad, que llama a Instalacion.estaDisponible (sin cruces con otras reservas) y además revisa que no haya una sesión de entrenamiento en esa instalación a esa hora. Si no está disponible, lanza ReservaNoDisponibleException.
- Si la instalación pertenece a un deporte individual, el club cuenta las reservas de esa disciplina que se cruzan con la nueva y le pregunta a DeporteIndividual.permiteNuevaReserva. Si ya se llegó al máximo de instalaciones simultáneas, lanza ReservaNoDisponibleException.
- Al final la Instalacion agrega la reserva y el club la guarda en su lista.

9. Persistencia

9.1 Cómo funciona

El enunciado pide guardar los datos en archivos dentro de una carpeta distinta a la de los fuentes. Por eso todo el estado del club se guarda en archivos de texto (.txt) dentro de la carpeta `datos`, que está en la raíz del proyecto y separada de `src`. La clase PersistenciaClub se encarga de esto: la consola la llama para cargar al arrancar y para guardar al salir (opción 0). Si un archivo no existe, simplemente se salta y esa parte arranca vacía.

Las reglas del formato son simples:

- Un archivo por tipo de objeto y una línea por objeto.
- Los campos de una línea van separados por punto y coma (`;`).
- Cuando un campo contiene una lista, sus elementos se separan con coma (`,`) o con barra (`|`).
- Las fechas se escriben como AAAA-MM-DD, las horas como HH:MM y los enums con su nombre (por ejemplo, ACTIVA).
- Un objeto que referencia a otro lo hace con una clave: el id del usuario, o el nombre del deporte, del equipo o del producto. En los partidos y las sesiones se usa el número de orden del objeto en su archivo.

9.2 Los 23 archivos

Archivo	Qué guarda cada línea
socios.txt	id; nombre; fecha de nacimiento; login; contraseña; puntos de fidelidad
empleados.txt	tipo (ENTRENADOR, FISIOTERAPEUTA o ADMINISTRADOR); id; nombre; fecha de nacimiento; login; contraseña; código de descuento

Archivo	Qué guarda cada línea
turnos.txt	id del empleado; día; hora de inicio; hora de fin
solicitudes_turno.txt	id del empleado; día y horas del turno original; nuevo día y nuevas horas; estado; id del administrador
deportes_conjunto.txt	nombre; mínimo de jugadores; posiciones; puntos por victoria; puntos por empate; si permite empate
deportes_individuales.txt	nombre; máximo de instalaciones
equipos.txt	deporte; nombre; categoría; cupo máximo; si está activo
inscripciones_equipo.txt	id del socio; deporte; equipo; posición; número de camiseta; estado
inscripciones_academia.txt	id del socio; deporte; nivel; puntaje de ranking; estado
competencias.txt	nombre; organizador; año; categoría; tipo; formato; deporte
inscripciones_competencia.txt	competencia; año; equipo; estado; nómina (ids de socios separados por coma)
partidos.txt	tipo (OFICIAL, INTERNO o PRACTICA); fecha; estado; marcador; datos propios del tipo separados por coma
estadisticas.txt	número del partido; id del jugador; tipo de estadística; valores
productos.txt	tipo (ARTICULO, BEBIDA o SNACK); nombre; precio; dato propio (categoría, si es caliente o alérgenos)
inventario.txt	producto; ubicación; cantidad
instalaciones.txt	tipo; id; capacidad máxima; si es techada; disciplina
movimientos.txt	fecha; cantidad; tipo; producto; origen; destino
pagos.txt	fecha; periodo; valor; id del socio
reservas.txt	id del socio; id de la instalación; fecha; hora de inicio; duración; número de jugadores; modalidad
sesiones.txt	número; fecha; hora de inicio; duración; id del entrenador; id de la instalación; tipo; equipo; deporte individual
asistencias.txt	número de la sesión; id del socio; estado; nota; comentario
sugerencias.txt	nombre; descripción; id del proponente; estado; id del administrador; producto

Archivo	Qué guarda cada línea
ventas.txt	fecha; id del comprador; tipo (TIENDA o CAFETERIA); si usó código compartido; si incluyó propina; productos y cantidades (producto,cantidad separados por barra)

9.3 Orden de carga

Al cargar, cada archivo necesita que ya existan los objetos que referencia, así que el orden importa: socios, empleados, solicitudes de turno, turnos, deportes de conjunto, deportes individuales, equipos, inscripciones de equipo, inscripciones de academia, estados de los equipos, competencias, inscripciones de competencia, partidos, estadísticas, productos, inventario, instalaciones, movimientos, pagos, reservas, sesiones, asistencias, sugerencias y ventas.

Al guardar se escribe cada archivo completo otra vez con el estado actual del club.

9.4 Por qué elegimos archivos de texto

La guía pide comparar tres criterios antes de elegir cómo guardar los datos:

- **Eficiencia en espacio:** el texto plano pesa más que un formato binario, pero el club maneja pocos datos (cientos de líneas), así que ese costo es pequeño.
- **Velocidad de búsqueda:** al arrancar se carga todo en memoria y las búsquedas se hacen sobre listas (por ejemplo, buscarUsuario recorre la lista). Es una búsqueda lineal, que con este volumen es rápida.
- **Usabilidad y mantenibilidad:** cada archivo se puede abrir y editar con un editor de texto, lo que sirve para preparar datos de prueba. Se leen y escriben con BufferedReader y BufferedWriter y se separan con split. Tener un archivo por tipo de objeto hace fácil encontrar un error.

Una base de datos relacional habría sido más robusta, pero era más de lo que necesitamos para este proyecto y se sale de lo que pide el enunciado.

9.5 Limitaciones

- Los textos no pueden contener `;`, `,` ni `|` porque no se escapan (la descripción de una sugerencia cambia `;` por `,` al guardar).
- En varios archivos, si una línea tiene menos campos de los esperados o referencia algo que no existe, esa línea se salta sin avisar. Si un valor de enum no es válido, la carga falla y la consola muestra el mensaje.
- Las ventas no guardan el precio, así que al cargarlas se usa el precio actual del producto.
- Las contraseñas quedan en texto plano.
- PersistenciaClub es una sola clase muy grande (unas 2.000 líneas) que conoce casi todas las clases del dominio.

10. Anexo: uso de inteligencia artificial generativa

Esta entrega se rige por el nivel 2 de uso de IA generativa: el uso se explica en este anexo e incluye la colección de prompts.

Herramienta usada en esta entrega: Claude (Anthropic).

Para qué la usamos:

- Comparar el UML y el informe de la Entrega 1 con el enunciado, la rúbrica y las correciones hechas por la monitora en la presentacion 1, asi hicimos una lista de correcciones.

Prompts principales de esta entrega:

1. “NO USES INFORMACION EXTERNA” (se uso solo el enunciado somo contexto, la rúbrica y nuestros entregables junto a lista de feedback y posibles correciones).
2. “revisa los siguientes documentos, la rubrica y el siguiente feedback para realizar una lista de errores y correciones a aplicar”